Proceedings of the 2026 Winter Simulation Conference
V. Ramamohan, A. Djanatliev, M. Fakhimi, C. Krejci, C. Ruiz Martin, B. S. Onggo, and N. Mustafee, eds.

A TUTORIAL ON GENERATIVE AI AND SIMULATION MODELING INTEGRATION

Mohammad Dehghani, Sahil Belsare, and Negar Sadeghi

Department of Mechanical and Industrial Engineering
Northeastern University
360 Huntington Ave, Boston, MA 02115, USA

ABSTRACT

This tutorial paper explores the applicability of Generative AI (GenAI), particularly Large Language Models, within the context of simulation modeling. The discussion is organized around three key perspectives: *Generation*, *Execution*, and *Analysis*. Building on the 2025 edition of this tutorial, the present revision contributes a fully reproducible code repository, an in-browser live demonstration, an updated agentic-AI workflow built on the Model Context Protocol (MCP), and an empirical comparison of frontier LLMs on the same simulation-generation prompt. The paper offers conceptual guidance and hands-on examples to support researchers and practitioners integrating GenAI into simulation environments.

1 INTRODUCTION

Some of the most revolutionary products appear to be conceived in a single night, yet they are almost always built on decades of quiet, accumulating research. The simulation modelling community knows this story well, and so does the artificial-intelligence community next door. Both fields have followed a strikingly parallel arc over the same six decades. Simulation went through a language era in the early 1960s with GPSS, SIMULA, SLAM, and SIMAN, then a GUI revolution in the 1990s with ProModel and Arena, then the multi-method platforms of the 2000s such as AnyLogic and Simio, and most recently an AI-augmented era that brings neural networks, reinforcement learning, and generative AI directly inside the modelling environment. Artificial intelligence followed the same staircase from a different starting point. The symbolic era ran from Dartmouth in 1956 through MYCIN in 1976. Statistical and neural methods took over from the late 1980s with backpropagation, support-vector machines, and random forests. The deep-learning revolution of the 2010s brought AlexNet and AlphaGo. Generative AI arrived in the early 2020s, with ChatGPT in 2022 as the public turning point (Figure 1). Each wave on each side reshaped what the technology could do. None arrived overnight. The point is not that 2025 was the year of generative AI. It is that 2025 was the year the two timelines started intersecting at the level of practitioner workflows.

The 2025 edition of this tutorial (Dehghani et al. 2025) introduced a three-perspective framework for thinking about that intersection in the context of discrete-event simulation. The three perspectives were Generation, Execution, and Analysis. The present paper is the continuation of that tutorial. It is updated for a year in which generative AI has reshaped what is possible at a speed without precedent in computing history. ChatGPT reached one hundred million monthly users within two months of launch, the fastest adoption curve ever recorded for a consumer technology (K. Hu 2023). Frontier models that struggled with discrete-event code in 2025 now produce working SimPy scripts on the first attempt. The Model Context Protocol (Anthropic 2024) has emerged as a standard way for language models to invoke external tools. And reproducible runnable artefacts, rather than purely conceptual tutorials, are increasingly what the community asks for.

This paper makes the following contributions:

- A revised tutorial on integrating generative AI into the discrete-event simulation lifecycle, updated to reflect frontier-model capabilities and tooling as of 2026.

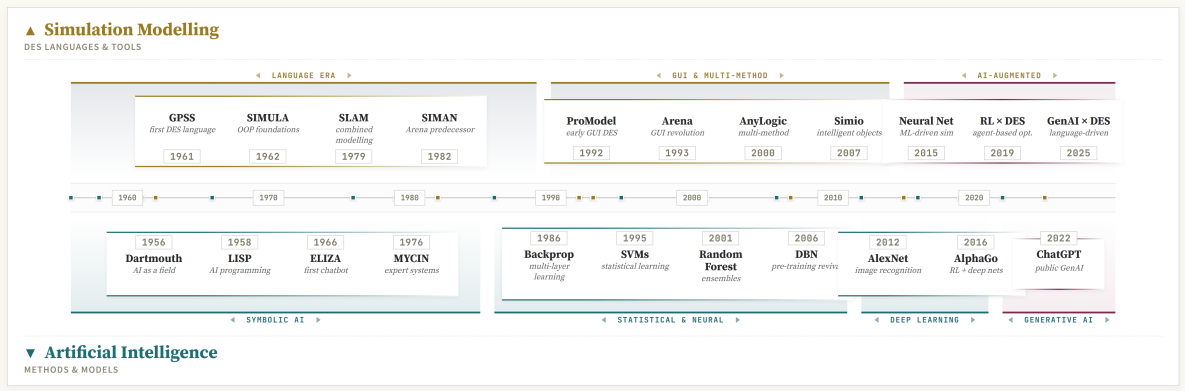


Figure 1: Parallel evolution of simulation modelling and artificial intelligence. The two timelines develop largely independently for half a century and begin to intersect in the practitioner workflow only in the 2020s.

- A working agentic-AI workflow built on the Model Context Protocol that allows language models to construct simulation models by calling exposed simulation primitives, rather than generating monolithic source files.
- An empirical comparison of three frontier large language models on the same simulation-generation prompt, scored on first-try success, code quality, and key-performance-indicator agreement.
- A fully reproducible accompaniment to the paper: a public source-code repository and an in-browser live demonstration of the running example, available at https://github.com/mdehghani86/wsc26-genai-sim and https://mdehghani86.github.io/wsc26-genai-sim/.

To ground these contributions in a concrete artefact, a multi-stage emergency department (ED) model is used as a single running case study throughout the paper. Patients arrive at the ED, are triaged into severity bands, may pass through imaging or laboratory services, and are then either admitted to inpatient beds or discharged. The case is intentionally rich enough to expose where current LLMs succeed and where they break. It requires branching, state tracking, multiple resource pools, and quantitative validation against analytical bounds. Section 3 defines the case in detail. Subsequent sections show how each stage of the GenAI-driven simulation workflow operates on it. The paper culminates in a side-by-side comparison of three implementation strategies in Section 9, namely vibe-coding, multi-agent generation, and MCP-driven tool use.

The rest of the paper is organised as follows. Section 2 provides a brief primer on large language models. Section 3 introduces the ED running example. Section 4 presents the three-phase GenAI-driven simulation framework. Sections 6, 7, and 8 elaborate the Creation, Execution, and Experimentation phases. Section 9 introduces the agentic-AI extension via the Model Context Protocol, Section 10 reports the empirical comparison of frontier LLMs, and Section 11 describes the reproducible artefacts. Section 12 concludes.

2 LARGE LANGUAGE MODELS IN SIXTY SECONDS

Generative AI has reshaped the technology landscape in the four years since ChatGPT's public release in late 2022, and at the centre of that change sits a single architecture, the large language model. Figure 2 places LLMs within the broader AI landscape. Artificial Intelligence is the umbrella. Machine Learning is a subset that learns from data. Deep Learning is a further subset that uses multi-layer neural networks. Natural Language Processing is the parallel discipline concerned with human language. LLMs are the present-day intersection of deep learning and NLP, doing both at unprecedented scale. The 2025 edition of

this tutorial (Dehghani et al. 2025) treated this material at full length. Here we keep only what is needed to read the rest of the paper, and refer the interested reader to that earlier treatment for the full version.

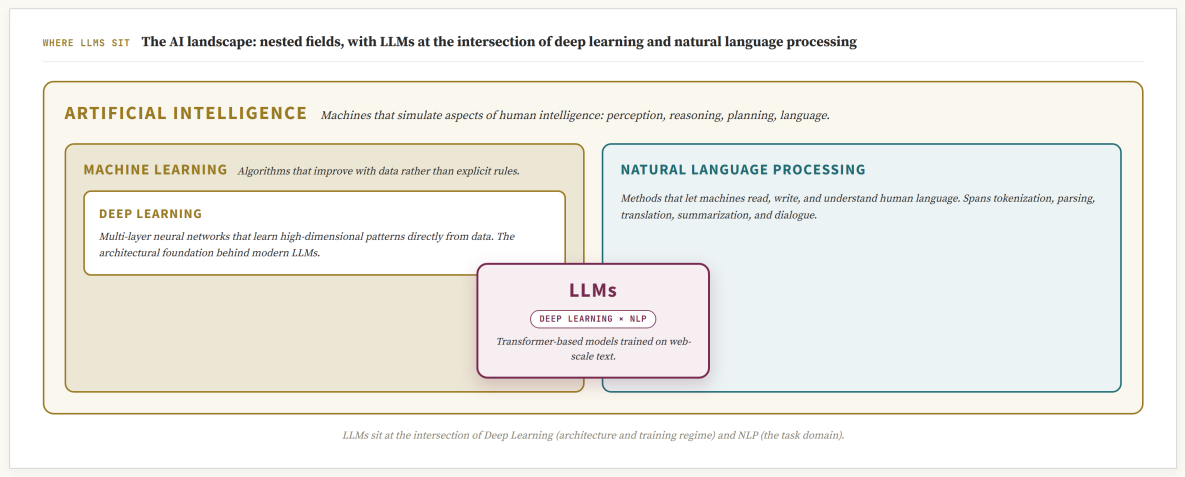
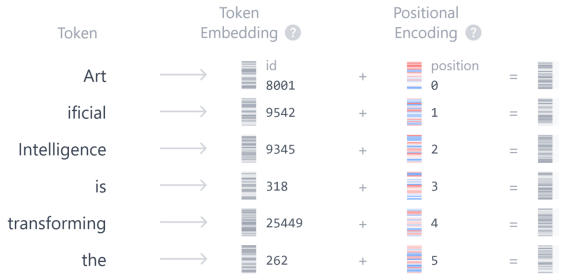


Figure 2: Where LLMs sit in the broader AI landscape: Artificial Intelligence contains Machine Learning and Deep Learning; LLMs sit at the intersection of Deep Learning and NLP.

2.1 How LLMs Work

An LLM predicts the next token in a sequence. The architecture that made the modern wave possible is the Transformer (Vaswani et al. 2017), which replaced the strictly sequential processing of older recurrent networks with self-attention. Self-attention lets the model weigh and relate every token in the input to every other token at once, regardless of position. Three steps run in order. First, the input string is tokenized and each token is mapped to a dense vector with a positional encoding added so that order survives (Figure 3a). Second, a stack of Transformer blocks repeatedly applies multi-head self-attention and feedforward layers, refining the contextual meaning of each token. Third, the final layer produces a probability distribution over the vocabulary, and the next token is sampled from that distribution (Figure 3b). The 2025 paper walks through the Query/Key/Value mechanics, the attention equation, and a worked example for readers who want the lower-level view.



(a) Token embedding and positional encoding.

(b) Output probability distribution over the next token.

Figure 3: First and last steps of a Transformer pass.

2.2 Controlling Output: Temperature and Top-k/Top-p

Because the next token is sampled from a probability distribution rather than chosen deterministically, the same prompt can produce different outputs on different runs. Three parameters control how much variation is allowed. *Temperature* rescales the distribution before sampling. Low temperature concentrates mass on the most likely token and produces near-deterministic output. High temperature flattens the distribution and produces more varied output (Figure 4). *Top-k* restricts the sampling pool to the k highest-probability tokens. *Top-p*, also known as nucleus sampling, keeps the smallest set of top tokens whose cumulative probability exceeds a threshold p, which adapts to the local shape of the distribution. Throughout this paper we run with temperature 0.2 for code-generation prompts and 0.7 for narration prompts, with top-p 0.9 in both cases. The 2025 paper documents the trade-offs in more detail.

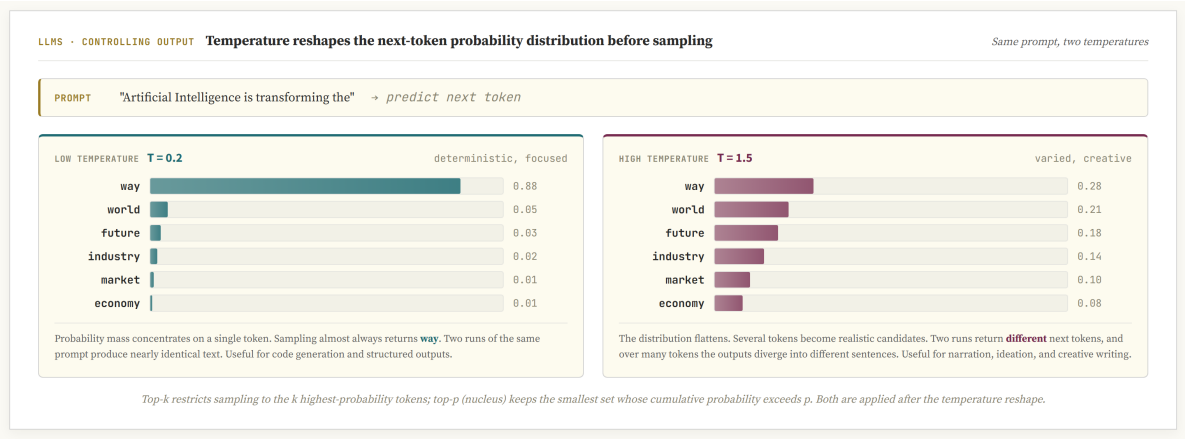


Figure 4: Token probabilities at low and high temperature: low temperature concentrates mass on the most likely token; high temperature flattens the distribution.

3 RUNNING EXAMPLE: EMERGENCY DEPARTMENT

4 GENAI-DRIVEN SIMULATION FRAMEWORK

The remainder of the paper organises GenAI-assisted simulation work into four typed stages (Figure 5). The 2025 edition of this tutorial used three stages and described the framework in narrative terms. The 2026 revision adds an explicit upstream stage, Problem Formulation, and tightens the contract between stages. Each stage produces a single artefact that the next stage consumes verbatim. Problem Formulation produces a three-row brief (problem, scope, KPIs). Creation produces a validated SimPy file together with its passing test report. Execution produces a run-time trace and the in-simulation decisions taken along the way. Experimentation produces a scenario table together with the figures and prose that interpret it. The contract between stages is what gives the workflow its discipline. A figure that does not trace back to a row in the brief is not part of the study. A scenario that the brief did not ask for is not reported. The five-check validation gate at the end of Creation is what makes downstream consumption safe. If the gate does not pass, no downstream stage runs.

5 PROBLEM FORMULATION

Before any code is written, the modeller has to settle what is being modelled and why. The first stage produces no code and no model file. It produces a written agreement on three items. The first is the problem to be solved. The second is the scope and boundaries of the system under study. The third is the set of key performance indicators (KPIs) against which the simulation will be judged. In a traditional

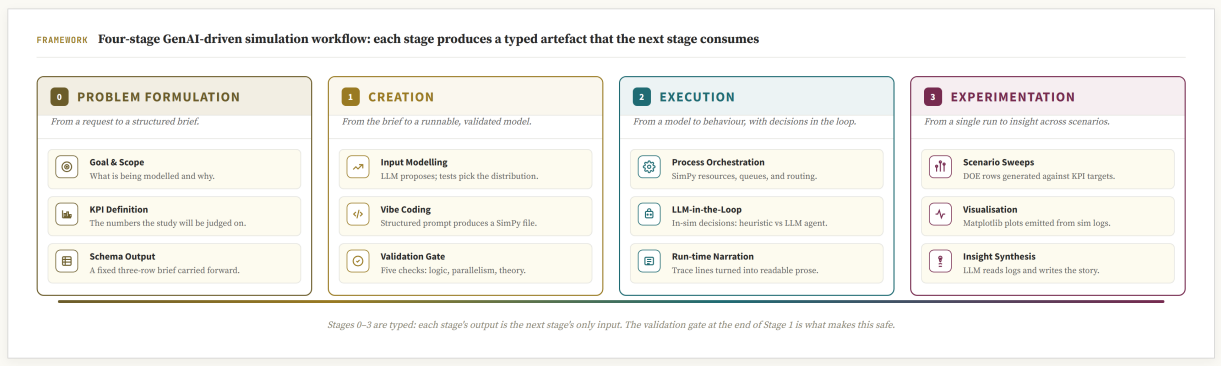


Figure 5: Four-stage GenAI-driven simulation workflow. Each stage produces a typed artefact that the next stage consumes: a structured brief, a validated SimPy file, an annotated trace, and a scenario interpretation. Stage colours match the section headings in the rest of the paper.

study these three items are spread across requirements meetings, white-board sketches, and the opening of a project brief. A recent analysis of professional GenAI use found ideation to be one of its most common roles (M. Zao-Sanders 2025). What we add here is structure. The conversation is the same, but the output is constrained to a fixed schema so that the artefact can be carried forward unchanged. All prompts shown in this paper were issued to Anthropic's Claude[1] and reproduced verbatim from the chat transcript.

Figure 6 shows the prompt and response for the ED running example. The prompt is doing three things at once. It states the request in natural language. It supplies a short note that anchors the model in the specific facility and study question. And it asks the LLM to consult recent literature so that the returned KPIs reflect community practice rather than the modeller's prior habits. The response is constrained to a fixed three-row schema. Forcing the structure has two practical effects. First, the output is directly comparable across projects and across language models. Second, it draws attention to whichever cell is weakest, which is almost always *Scope and boundaries*, and lets that conversation happen up front rather than after a two-hundred-line script has already been written against an unstated assumption. The caveat that Sun et al. raised about LLM ideation in general applies here (C. Si and D. Yang and T. Hashimoto 2024). The model is recombining published practice, not inventing a new study design. The structured schema is what turns that recombination into a usable artefact.

The three rows of the response are not a deliverable in isolation. They are the inputs to the three downstream stages. *Scope and boundaries* determines which resources and processes appear in the SimPy file generated by Creation (Section 6). If the row excludes ambulance routing, no SimPy resource is created for it and no narration mentions it later. The *problem definition* in the first row is the question that Execution (Section 7) must ultimately answer. A trace that does not address it has run, but has not produced evidence. The *KPIs* in the third row become the success criteria of Experimentation (Section 8). Every reported number, every plot, and every scenario comparison is judged against this row.

6 CREATION

The Creation stage takes the three-row brief and produces a runnable, validated SimPy file. It has two activities. The first is input modelling, which turns the *Scope* row into the random-variate generators the model will need. The second is model generation by structured prompting, with a five-check validation gate that decides whether the file is fit to travel forward.

[1]Specifically, claude-opus-4-7. Section 10 reports an empirical comparison against two other frontier models on the same simulation-generation prompt.

PROMPT

I am planning a discrete-event simulation of an emergency department. Help me formulate the study as a structured problem statement.

NOTES small community hospital, two triage tiers, the team wants to evaluate staffing and routing rule changes.

SEARCH recent literature (2023–2025) on emergency-department simulation studies and pull common KPIs.

RETURN a single table with three rows: *Problem definition*, *Scope & boundaries*, and *KPIs*. Keep each cell to two or three short sentences.

RESPONSE

	ELEMENT	STATEMENT
1	Problem definition	Reduce length of stay in the emergency department and identify whether triage-rule changes or staffing changes deliver the larger improvement under fluctuating arrival rates.
2	Scope & boundaries	From walk-in arrival to admit-or-discharge. Two triage tiers (urgent, non-urgent), a shared imaging and laboratory pool, and fixed bed pools. Ambulance routing, no-shows, and inpatient bed flow are out of scope.
3	KPIs	Mean and 95th-percentile length of stay; door-to-doctor time per triage tier; bed and imaging utilisation; daily throughput.

Figure 6: Problem-formulation prompt and response: a fixed three-row schema (problem, scope, KPIs) that downstream stages consume unchanged.

6.1 Input Modelling

The 2025 edition of this tutorial framed input modelling as one of several activities the LLM could help with, and the recent literature has pushed further in that direction. SimLLM (J.-Q. Chen and K. Zhang and R. Zheng and Y. Zhong 2026) fine-tunes code LLMs specifically on SimPy queueing data, and LLM-driven DES frameworks delegate input fitting to multi-agent pipelines (M. Selak and D. Krechel and A. Ulges and S. Spieckermann and N. Stoehr and A. Loehr 2026). Both are powerful, but both also keep the language model as the final arbiter of the distribution choice. We take a deliberately narrower position. Goodness-of-fit is a solved statistical problem. Distribution-free tests have been understood since Anderson and Darling (1952) and packaged in `scipy.stats` for two decades (Delignette-Muller and Dutang 2015). An LLM that picks the distribution by inspection is throwing away that work and adding a non-reproducible step in its place. We use the LLM only at the two ends of the pipeline, proposing candidates and writing the verdict, and give the test the final word in between.

The pipeline is short. The user supplies a numeric column or timestamps. After basic checks (sample size at least 30, no negative values, zeros flagged) the data are summarised numerically (size, mean, standard deviation, coefficient of variation, skewness). The LLM is prompted with that summary and is asked for two or three candidate continuous distributions, one rationale each, and *no fit*. The candidates are then fitted with `scipy.stats`, and Kolmogorov–Smirnov, Anderson–Darling, and a binned chi-square test are run against the fitted CDFs. Figure 7 shows the result on a deliberately bursty arrival trace where all three of the LLM's candidates are rejected at the 5% level. The empirical histogram and the LLM proposals look reasonable in isolation, but the Q-Q plot and the test column make the failure visible. The data are non-stationary (rush hours mixed with off-peak), and no single marginal distribution can absorb that. The verdict at the bottom of the figure is what the LLM writes after reading the test output. The recommendation it produces, refit per hour-of-day, is what the modeller would have reached with classical input-modelling practice. The point is not that the LLM is wrong, but that the test is what made the failure visible.

The artefact produced by input modelling is a single Python expression: a callable that returns one inter-arrival sample, together with a banner noting whether the test accepted the fit or whether the closest-rejected candidate was used as a fallback. The model-generation step (Section 6.2) consumes that expression as the arrival generator. If the banner reports a rejection, the prompt is instructed to refuse the global fit and to ask for hour-of-day stratification instead. The connectivity is therefore mechanical, not narrative. The brief row that asked for "arrival rate" is answered by a Python line that the next step reads, and the goodness-of-fit verdict travels with it.

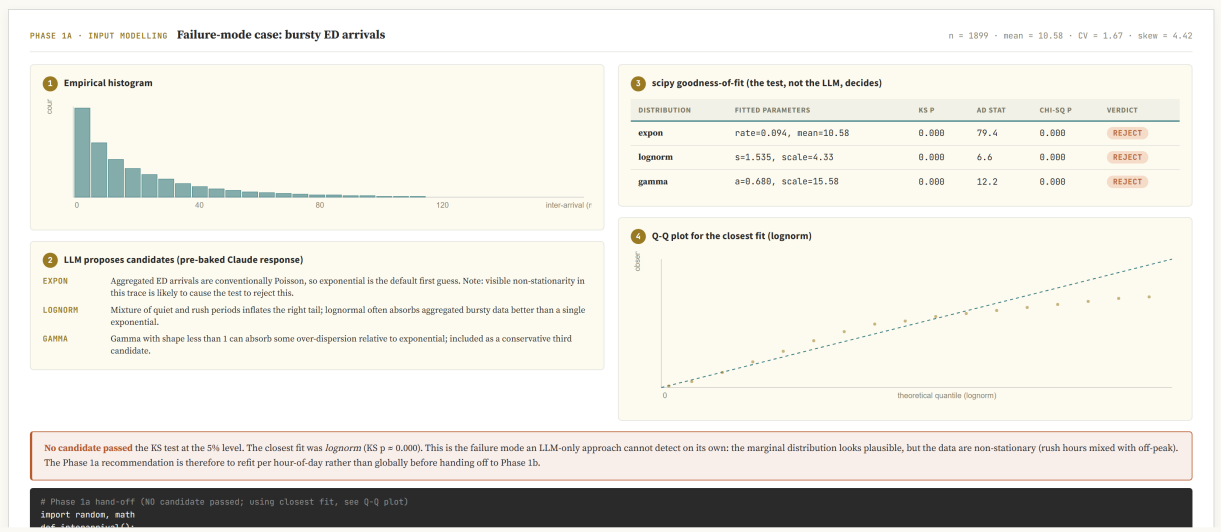


Figure 7: Input modelling on a deliberately bursty ED arrival trace. The LLM proposes three candidate distributions with one-line rationales; `scipy.stats` fits each and runs KS, AD, and chi-square goodness-of-fit. All three candidates are rejected at the 5% level. The Q-Q plot shows systematic deviation in the upper tail, and the verdict (written by the LLM after reading the test output) recommends refitting per hour-of-day.

6.2 Model Generation: Structured Prompting with a Validation Gate

Off-the-shelf simulation packages have offered template-based modelling for two decades. Tools such as Simio, Arena, FlexSim, and AnyLogic let the modeller drag a server, a queue, or a routing decision onto a canvas, configure parameters in dialogs, and publish a runnable model without writing code. The template editor enforces invariants by construction. A server has a capacity. A queue feeds exactly one server. A routing decision lists every outgoing arc. The parameter dialog refuses out-of-domain values. The question we ask in this section is whether the same invariants can be enforced when the model is generated as Python code rather than placed on a canvas. The answer in this paper is yes, but only if the invariants are made an explicit automated check rather than left for the modeller's eye.

The 2025 edition introduced a five-part structured prompt for SimPy generation, comprising Role, Goal, Scenario, Inputs, and Output. We retain the structure but feed it from the upstream artefacts. The Role asks the LLM to write as a simulation engineer. The Goal is to produce a runnable SimPy file for the stated problem. The Scenario is the *Scope and boundaries* row, copied verbatim, so the LLM cannot drift into adjacent ED variants the modeller did not ask for. The Inputs section pastes the arrival generator from input modelling together with the fitted distributions for service times. The Output section asks for one Python file with no dependencies beyond SimPy and the standard library. SimLLM (J.-Q. Chen and K. Zhang and R. Zheng and Y. Zhong 2026) has shown that fine-tuned 7B-class models can generate executable SimPy on a similar prompt template, and M. Selak and D. Krechel and A. Ulges and S. Spieckermann and N. Stoehr and A. Loehr (2026) reach the same conclusion via a multi-agent pipeline. Both confirm that frontier-model zero-shot prompting is a defensible default for a tutorial reader.

Frontier LLMs are powerful but probabilistic. On the same prompt, two runs can produce structurally different SimPy files, and silent bugs in the generated code can survive a casual read. We therefore put a fixed five-check validation gate between the generated file and the downstream stages (Figure 8). Process logic checks that the patient flow matches the brief end to end. Connectivity checks that each producer wires into the declared consumer with no orphan resources. Parallel-versus-sequential checks that a pool with capacity c actually serves c patients in parallel. This is one of the silent failure modes that motivated the WSC

2025 lessons-learned review of this case study. A single per-pool "manager" process collapses effective parallelism to one even when `simpy.Resource(capacity=c)` is declared correctly. Distribution checks verify that sampled service times match the input-modelling fit in mean, variance, and support. Theory checks compare realised utilisation, mean queue length, and mean waiting time against closed-form Erlang-C and Allen–Cunneen predictions for the matching $M/G/c$ system. If any check fails, the failing check is appended to the next prompt verbatim and the LLM is asked to regenerate only the affected section. If three rounds of regeneration do not pass, the case is handed off to the multi-agent or MCP path of Section 9. The artefact that leaves Creation is therefore not just a SimPy file. It is a SimPy file plus a passing validation report. Either both travel forward, or neither does.

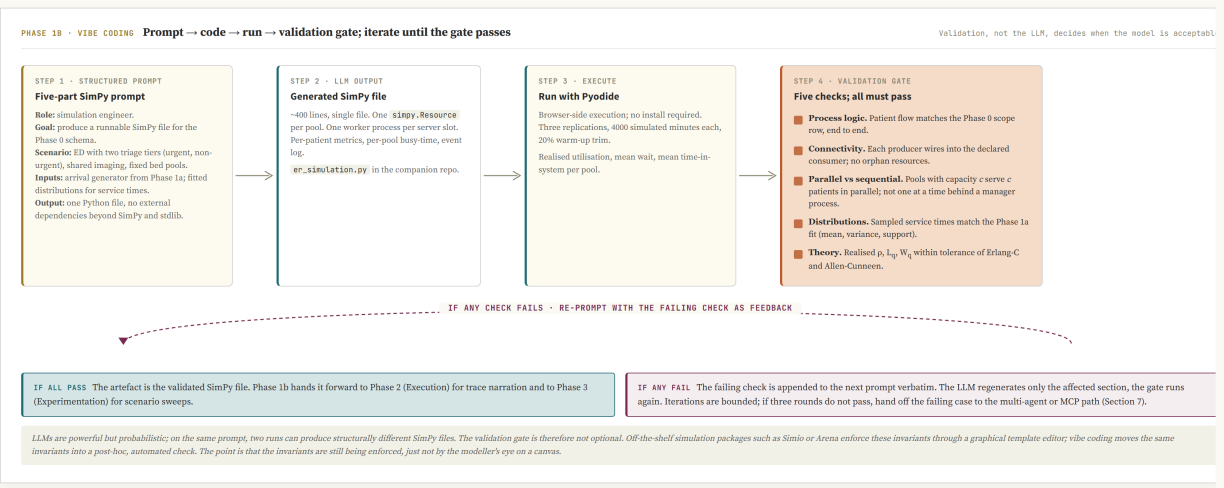


Figure 8: Vibe-coding loop with a five-check validation gate (process logic, connectivity, parallelism, distributions, theory). If any check fails, the failing check is appended to the next prompt and iterations are bounded. Off-the-shelf template editors enforce the same invariants at draw time; the gate is the code-based equivalent.

The validated SimPy file is published as `phase1b/er_simulation.py` in the companion repository (Section 11). Execution (Section 7) reads it as input. Experimentation (Section 8) runs scenario sweeps against it and scores them against the KPI list.

7 EXECUTION

The Execution stage takes the validated SimPy file and runs it. Two questions arise at this point that the 2025 edition only touched on. The first is what to do with a decision point inside the simulated system. Bed assignment, lane routing, and dispatching are decisions that happen inside the simulation as it runs, not before it starts. A traditional model fixes the decision rule by hand. A GenAI-assisted model can call out to an LLM at every decision point and let the language model decide. The second question is what the run-time output of the simulation should look like. A table of timestamps and resource holds is what SimPy produces by default, but it is not what a stakeholder wants to read.

7.1 In-Simulation Decisions: Heuristic versus LLM-in-the-Loop

The ED running example has a recurring decision point. Each arriving patient must be assigned to one of b inpatient beds (or held in queue if all are busy). The simplest heuristic is round-robin: assign cyclically by bed index. A slightly stronger rule prioritises by severity, then breaks ties by time-in-queue. A clinician-trained rule looks at remaining length-of-stay estimates and tries to balance bed turnover against acuity. All three are deterministic functions of the current system state.

We add a fourth option. At each assignment step the simulation pauses, serialises the relevant state (current bed occupancy, severity, time-in-queue, expected length of stay), sends it to an LLM with a short instruction, and uses the returned bed index. Recent work has demonstrated this pattern in adjacent domains. Y. Quan and Z. Liu (2024) place an LLM agent inside a multi-echelon inventory simulation as the order-quantity decider, and He and Bian (2025) place one inside a transportation simulation as the route-choice decider. Pseudocode for our setting is short:

```
def llm_assign(state, llm):
  prompt = render_prompt(state)
  reply  = llm(prompt, temperature=0.0, max_tokens=8)
  return parse_bed_index(reply)
```

The Execution stage is therefore a place where the simulation engine and the language model trade control. At every decision point the engine hands over the state, the model returns a choice, and the engine resumes. Three properties follow. First, latency matters. A frontier-model call adds tens of milliseconds to seconds of wall-time per decision, which is not free in a study with 10^5 assignments. The companion application caches identical state vectors and batches calls in trace replay so that the cost stays within tutorial budgets. Second, reproducibility matters. We fix temperature to zero and record the prompt-and-reply for every decision so the run can be replayed. Third, the comparison is the point. Running the same trace through round-robin, severity-priority, the clinician-rule baseline, and the LLM gives four KPI columns judged against the same brief. The agreement rate between LLM and clinician-rule is the diagnostic. Where they agree, the LLM is doing what a sensible rule would do; where they disagree, the disagreement is the place to read the prompts.

For comparability across vendors, the Execution stage keeps a small driver that abstracts the model call into a uniform interface. The same loop runs with Claude, Gemini, or GPT behind it. The companion application exposes the four heuristics as built-in baselines. To exercise the LLM-in-the-loop variant the user supplies their own API key for whichever vendor they prefer. We did not embed shared keys in the public site for cost reasons. The keys never leave the browser tab. The point of the section, however, is methodological rather than UI-shaped. A simulation engine that can call out to a language model at decision time is a different artefact from one whose decision rule is fixed at compile time, and the methodological consequences (latency, cost, reproducibility, agreement-rate diagnostics) are what we care about.

7.2 Run-time Narration

The default SimPy trace is a stream of `(time, entity, event)` tuples. It is unreadable in raw form. The 2025 edition introduced LLM-driven trace narration, prompting the model with a sliding window of trace lines and asking for a short paragraph of plain English. The same approach carries over for the 2026 revision, with two refinements. First, the prompt now includes the brief row that the trace must address (so the narration is purposive rather than a transcript). Second, the narration is grouped by entity (patient) rather than by clock time, which lets the reader follow one patient end to end instead of reconstructing journeys from interleaved events.

8 EXPERIMENTATION

The Experimentation stage takes the validated model, sweeps it across scenarios, and turns the results into something a reader can act on. It has three activities. The first is the sweep itself: which factors vary, at what levels, with how many replications. The second is visualisation: turning the resulting numbers into figures that support comparison rather than mere display. The third is insight synthesis: turning the figures and the table into a short narrative that addresses the brief.

8.1 Scenario Sweeps

The factors and levels are not invented from scratch. They are read off the brief. The KPI row tells us what to measure (e.g., mean length of stay, 95th-percentile waiting time, bed-utilisation). The scope row tells us what is movable in the system under study (e.g., bed count, triage staffing, hour-of-day arrival pattern). A small full-factorial or fractional-factorial design over the movable factors, replicated to the precision the KPI row demands, produces the data table. The LLM contribution at this stage is bounded and explicit. It proposes a starting design from the brief, the modeller accepts or trims it, and the rest is mechanical: generate the scenario file, run, collect.

8.2 Visualisation from Simulation Logs

SimPy logs are amenable to direct plotting. The companion artefact ships a small `viz.py` that reads the standard log format and emits matplotlib figures: KPI bar charts across scenarios, queue-length time series with confidence bands, utilisation heatmaps by resource and shift, and length-of-stay distributions by severity. The role of the LLM here is figure selection rather than figure drawing. Given the KPI list and the scenario table, it is asked which two or three plots best support the comparison the brief calls for. Because the plots are produced by code rather than by the LLM, what the LLM actually emits is a sequence of `viz.py` function calls. The figures themselves are reproducible from the log file alone.

8.3 Insight Synthesis

The final activity is a short written interpretation. The 2025 edition used a third-party tool (Napkin) to generate visual summaries from log text. The 2026 revision keeps the spirit (LLM as the writer of the report) but moves the activity inside the workflow. The LLM is given three things: the KPI rows from the brief, the scenario table, and short captions for the plots. It is asked to write a paragraph addressed to a stakeholder, naming which scenario performs best on which KPI and where the trade-offs lie. The output is reviewed by the modeller before it leaves the loop. The reason for keeping a human in this seat is the same as in input modelling. The model is good at recombining what it sees, and that is exactly the right capability for synthesising a written summary, but the responsibility for the claim rests with the modeller. Recent work on LLM-assisted result interpretation in adjacent domains supports this division of labour, with the model handling first-draft narrative and the analyst keeping the final word (M. Selak and D. Krechel and A. Ulges and S. Spieckermann and N. Stoehr and A. Loehr 2026; J. Kleiman and K. Frank and S. Campagna 2025).

9 VIBE CODING VERSUS AGENTIC AI VIA MODEL CONTEXT PROTOCOL

The validation gate of Section 6.2 catches model files that fail invariants the LLM was asked to respect. It does not change the failure mode itself. The LLM produces a complete file in one shot, the gate either passes it or rejects it, and a rejection sends the entire file back for regeneration. This is acceptable for tutorial-scale models, but it scales poorly. A failure on one resource regenerates the whole file. A small change to the brief regenerates the whole file. The diagnostic information is the failed check, not the offending line. This section asks whether a different interaction pattern, in which the LLM is constrained to construct the model through a small set of typed actions rather than emit a Python file, gives back what one-shot vibe-coding loses.

9.1 What MCP Is, and Why It Is the Right Abstraction Here

The Model Context Protocol is an open standard, introduced by Anthropic in November 2024, for connecting language models to external tools and data sources (Anthropic 2024). An MCP *server* exposes three primitives: tools (functions the LLM can call), resources (read-only data the LLM can request), and prompts (parameterised templates). An MCP *client*, embedded in the LLM host application, discovers the

server's capabilities at session start and routes the model's tool calls to the server over JSON-RPC. The protocol has been the subject of two recent surveys, one focused on MCP itself (A. Singh and A. Ehtesham and S. Kumar and T. Talaei Khoei 2025) and one positioning it within the broader landscape of agent interoperability protocols (A. Ehtesham and A. Singh and G. K. Gupta and S. Kumar 2025).

The relevance to simulation is direct. A SimPy model is a small set of declarations and a small set of process definitions. If those operations are exposed as MCP tools, the LLM no longer writes a Python file. It calls `define_resource`, `define_process`, `wire`, `run`, and `query_kpis` in sequence, and the server enforces the invariants on each call. A capacity argument that is not an integer is rejected at the call. A wire that points at an undeclared consumer is rejected at the call. Theory checks run after `run` and report a diagnostic that the LLM can read and act on without regenerating any code. The construction of the model becomes a structured dialogue with a typed API, rather than a bet on a one-shot Python emission.

9.2 Tool Catalogue and Construction Trace

The companion repository ships a minimal `simpy-mcp` server that exposes five tools: `define_resource(name, capacity)`, `define_process(name, generator_body)`, `wire(producer, consumer)`, `run(duration, replications, seed)`, and `query_kpis(group_by=...)`. Construction of the ED model proceeds by sequence. The LLM reads the brief, calls `define_resource` for triage, imaging, lab, and beds, calls `define_process` for arrivals, triage, and admit-or-discharge, and `wires` the producers to the consumers. `run` returns a structured result. `query_kpis` produces the rows the brief asked for. If a wire is wrong, the server returns a typed error and the LLM corrects the call. If a theory check disagrees with the realised utilisation, the server returns the disagreement and the LLM revises only the relevant parameter.

9.3 Where Each Path Wins

Vibe coding, with the validation gate of Section 6.2, is the right path when the modeller wants a single file they can read, version, and run outside the loop. The output is a self-contained `.py` that anyone with SimPy installed can execute. The MCP path is the right path when the model is built incrementally, when invariants must be enforced at construction time rather than after the fact, or when the same brief will be re-run against shifting parameter sets. The two paths are not in opposition. The companion repository ships both, and the framework figure (Figure 5) shows them as alternative routes through the Creation stage. Recent work in this direction independently confirms the trade-off: Vasa and Sarjoughian (2025) take the structured-construction route via PDEVS-LLM with a statechart-driven workflow, and J. Kleiman and K. Frank and S. Campagna (2025) present a general simulation-agent framework that decouples natural-language scenario configuration from model execution. Our contribution is the side-by-side: the same brief, the same KPIs, two routes, and a comparison whose dimensions are agreed in advance (iteration count, first-pass executability, diagnosability of failures).

10 EMPIRICAL COMPARISON OF FRONTIER LLMS

11 REPRODUCIBILITY AND LIVE DEMONSTRATION

12 CONCLUSION

ACKNOWLEDGMENTS

REFERENCES

Anderson, T. W., and D. A. Darling. 1952. "Asymptotic Theory of Certain "Goodness of Fit" Criteria Based on Stochastic Processes". *Annals of Mathematical Statistics* 23(2):193–212.

Anthropic 2024, November. "Introducing the Model Context Protocol". Anthropic Engineering Blog. Accessed April 2026.

J.-Q. Chen and K. Zhang and R. Zheng and Y. Zhong 2026. "SimLLM: Fine-Tuning Code LLMs for SimPy-Based Queueing System Simulation". arXiv preprint arXiv:2601.06543.

Dehghani, M., S. Belsare, and N. Sadeghi. 2025. "A Tutorial on Generative AI and Simulation Modeling Integration". In *Proceedings of the 2025 Winter Simulation Conference*, 1197–1211. Seattle, WA.

Delignette-Muller, M.-L., and C. Dutang. 2015. "`fitdistrplus`: An R Package for Fitting Distributions". *Journal of Statistical Software* 64(4):1–34.

A. Ehtesham and A. Singh and G. K. Gupta and S. Kumar 2025. "A Survey of Agent Interoperability Protocols: Model Context Protocol (MCP), Agent Communication Protocol (ACP), Agent-to-Agent Protocol (A2A), and Agent Network Protocol (ANP)". arXiv preprint arXiv:2505.02279.

He, P., and W. Bian. 2025. "Agentic Large Language Models for Day-to-Day Route Choices". *Transportation Research Part C: Emerging Technologies* 178:105311.

K. Hu 2023, February. "ChatGPT Sets Record for Fastest-Growing User Base—Analyst Note". Reuters. Accessed April 2026.

J. Kleiman and K. Frank and S. Campagna 2025. "Simulation Agent: A Framework for Integrating Simulation and Large Language Models for Enhanced Decision-Making". arXiv preprint arXiv:2505.13761.

Y. Quan and Z. Liu 2024. "InvAgent: A Large Language Model based Multi-Agent System for Inventory Management in Supply Chains". arXiv preprint arXiv:2407.11384.

M. Selak and D. Krechel and A. Ulges and S. Spieckermann and N. Stoehr and A. Loehr 2026. "Agentic Insight Generation in VSM Simulations". arXiv preprint arXiv:2604.12421.

C. Si and D. Yang and T. Hashimoto 2024. "Can LLMs Generate Novel Research Ideas? A Large-Scale Human Study with 100+ NLP Researchers". arXiv preprint arXiv:2409.04109.

A. Singh and A. Ehtesham and S. Kumar and T. Talaei Khoei 2025. "A Survey of the Model Context Protocol (MCP): Standardizing Context to Enhance Large Language Models". Preprints.org 202504.0245.

Vasa, V. K. S., and H. S. Sarjoughian. 2025. "Generative Statecharts-Driven PDEVS Modeling". In *Proceedings of the 2025 Winter Simulation Conference*. Code at https://github.com/comses/PDLM.

Vaswani, A., N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, *et al*. 2017. "Attention Is All You Need". In *Advances in Neural Information Processing Systems 30 (NeurIPS)*, 5998–6008.

M. Zao-Sanders 2025, April. "How People Are Really Using GenAI in 2025". Harvard Business Review. Accessed April 2026.

AUTHOR BIOGRAPHIES

MOHAMMAD DEHGHANIMOHAMMADABADI is an Associate Teaching Professor in the Department of Mechanical and Industrial Engineering at Northeastern University. His research focuses on developing intelligent simulation frameworks integrated with optimization and AI-based models to support decision-making, with applications in healthcare, manufacturing, and supply chain. His email address is m.dehghani@northeastern.edu.

SAHIL BELSARE is a simulation scientist with expertise in digital twin technologies and simulation modeling. He applies machine learning models to enhance decision-making in complex systems, focusing on optimization and reinforcement learning integrated with simulation platforms. His email is belsare.s@northeastern.edu.

NEGAR SADEGHI is a Ph.D. student in Industrial Engineering at Northeastern University. Her research focuses on developing intelligent simulation models for pharmaceutical supply chains and production-distribution systems. Her email address is sadeghi.ne@northeastern.edu.